PRE-DECISIONAL — FOR TECHNICAL REVIEW ONLY

AAIS Doctrine Module v2.1

Governed Autonomy for Mission-Critical Systems

Executive Technical Summary — Prepared for NASA Autonomous Systems Review

Prepared by: Cognitive Operating Systems Division, AAIS Platform

Date: April 2026

Document Version: 2.1.0-ETS

Distribution: NASA Autonomous Systems Division · JPL Autonomy Teams · Mission Assurance Reviewers

---

AAIS Doctrine Module — One-Page Overview

The Problem

Autonomous systems operating in deep space, planetary surfaces, and orbital construction environments face a fundamental constraint: communication latency and hardware degradation make real-time ground control impossible. These systems must make consequential decisions without human oversight — but unconstrained autonomy in mission-critical environments is unacceptable. The AAIS Doctrine Module solves this by enforcing governed autonomy: adaptive behavior within codified, non-negotiable behavioral boundaries.

Three Pillars of Governed Autonomy

- **Swarm Law** — Governs multi-agent coordination through codified consensus rules and escalation protocols. No single agent can override collective governance. Deadlocks default to the most conservative safe action.
- **Identity and Role Enforcement** — Every agent's identity is cryptographically anchored and continuously verified. Role boundaries are

enforced at the architectural level — agents cannot exceed their assigned capabilities, even under pressure or novel conditions.

- **Degradation Doctrine** — Defines pre-mapped fallback chains for every failure mode. The system always transitions to a known-safe state. "Fail-to-silent" is prohibited. Recovery paths are defined alongside degradation paths.

Architecture

[DIAGRAM: Doctrine Module — Simplified Architecture]

Three-layer horizontal stack. Top: Agent Layer (multiple agent instances proposing actions). Middle: Doctrine Module (governance constraint surface — all actions pass through). Bottom: Invariant Engine + Memory Subsystem + Hardware Abstraction. Right side: Operator Interface with bidirectional arrows to Doctrine Module. Single flow arrow: Agent proposes action → Doctrine evaluates → Invariant Engine verifies → action executes or is blocked.

Scenario: Mars Multi-Rover Coordination

Three rovers operating on the Martian surface detect a high-value geological formation. Rover-2 proposes redirecting all units to the site. Swarm Law requires consensus — Rover-1 and Rover-3 evaluate the proposal against their local doctrine state and mission priorities. Rover-3 votes non-compliant: the redirect would violate a power-reserve invariant for its current battery level. The Doctrine Module blocks the proposal and escalates. The swarm recalculates: Rover-1 and Rover-2 redirect while Rover-3 continues its assigned transect at reduced power. Every decision is logged, traceable, and auditable. No rover improvised. No governance rule was violated.

Table of Contents

1. Executive Summary
2. Architecture Overview
3. Pillar I — Swarm Law
4. Pillar II — Identity and Role Enforcement
5. Pillar III — Degradation Doctrine
6. Invariant Engine — Constraint Enforcement Layer
7. Governance Integration and Auditability
8. Comparative Advantage
9. Conclusion and Recommended Next Steps

1. Executive Summary

The AAIS Doctrine Module v2.1 is the governance backbone of the Autonomous Adaptive Intelligence System (AAIS) platform. It enforces behavioral invariants, identity boundaries, and operational continuity across multi-agent autonomous architectures deployed in mission-critical environments. This document provides a technical summary of the module's architecture, core pillars, and integration characteristics for evaluation by NASA reviewers.

The Doctrine Module was designed to address a fundamental challenge in autonomous space systems: environments where communication latency, hardware degradation, and unpredictable conditions render traditional ground-controlled operational paradigms insufficient. In deep-space missions, planetary surface operations, and degraded-communications scenarios, autonomous systems must make consequential decisions without real-time human oversight. The Doctrine Module ensures that those decisions remain within rigorously defined governance boundaries — regardless of environmental conditions.

The module is built on three core pillars:

- **Swarm Law** — Governs coordination, consensus, and conflict resolution among multiple autonomous agents operating as a collective.

- **Identity and Role Enforcement** — Ensures every agent maintains a stable, verifiable identity and operates strictly within assigned role boundaries.
- **Degradation Doctrine** — Defines governed degradation paths so the system always transitions to known-safe states rather than undefined or emergent failure modes.

Together, these pillars provide a governance framework that enables adaptive autonomy within codified behavioral constraints. The system can learn, adapt, and respond to novel conditions — but it cannot violate its foundational governance laws. This architecture addresses NASA's requirement for autonomous systems that are simultaneously capable and predictable, flexible and auditable.

> **Key Takeaway**
>
> The Doctrine Module is not advisory middleware — it is a foundational constraint surface. All agents in the AAIS architecture must comply with its governance laws at runtime. Non-compliance triggers automatic escalation and, if unresolved, agent isolation.

2. Architecture Overview

The Doctrine Module occupies a privileged position within the AAIS cognitive architecture. It functions as a **runtime governance layer** that sits between the agent execution environment and the platform's core subsystems. Every agent action, inter-agent communication, and state transition is subject to doctrine evaluation before execution.

The module interfaces with four primary architectural components:

- **Agent Layer** — The population of autonomous agents executing mission tasks. Each agent's behavioral output passes through doctrine compliance checks before being actuated.

- **Invariant Engine** — The formal verification subsystem that evaluates agent actions against codified behavioral invariants. The Doctrine Module supplies the invariant definitions; the Invariant Engine performs runtime evaluation.
- **Memory Subsystem** — The persistent state store for agent memory, mission context, and governance event logs. The Doctrine Module reads from and writes to this subsystem to maintain governance continuity across operational sessions.
- **Operator Interface** — The ground-side control surface through which human operators configure governance profiles, review compliance logs, and issue doctrine overrides when communication permits.

[DIAGRAM: AAIS Doctrine Module — Architectural Position within Cognitive Stack]

This diagram should depict the Doctrine Module as a horizontal governance layer spanning the full width of the cognitive stack. Above it: the Agent Layer (multiple agent instances). Below it: the Invariant Engine, Memory Subsystem, and Hardware Abstraction Layer. To the right: the Operator Interface with bidirectional communication arrows. All agent outputs pass downward through the Doctrine Module before reaching actuators. Governance event logs flow from the Doctrine Module into the Memory Subsystem.

The Doctrine Module codifies behavioral laws that are **enforced at runtime**, not merely suggested or advisory. This distinction is critical: in conventional autonomous system architectures, governance rules are often implemented as soft constraints or heuristic preferences that can be overridden by sufficiently high-priority objectives. In the AAIS architecture, doctrine laws are hard constraints — they cannot be violated by any agent, regardless of priority level, without triggering the escalation protocol defined in Swarm Law.

Governance enforcement operates at three temporal scales:

Enforcement Scale	Frequency	Description
Pre-Action	Every agent action	Each proposed agent action is evaluated against doctrine invariants before execution. Non-compliant actions are blocked and logged.
Periodic Audit	Configurable interval (default: 60s)	The Doctrine Module performs a full governance state audit, verifying agent identities, role boundaries, and swarm coherence.
Event-Triggered	On state transitions	Major events (agent join/leave, degradation level change, communication loss) trigger immediate doctrine evaluation.

Key Takeaway

The Doctrine Module is not optional middleware. It is a foundational constraint surface embedded at the architectural level. Removing or bypassing it would require dismantling the cognitive stack itself — by design.

3. Pillar I — Swarm Law

3.1 Purpose

Swarm Law governs coordination, consensus, and conflict resolution among multiple autonomous agents operating as a swarm or cooperative team. It defines the rules by which agents reach collective decisions, resolve disagreements, and maintain operational coherence — even under adverse conditions including partial agent loss, communication disruption, and environmental uncertainty.

3.2 Core Principles

Swarm Law is built on four foundational principles:

ID	Principle	Definition
SL-1	No Unilateral Override	No single agent may override collective governance without invoking the escalation protocol. Individual agent authority is bounded by swarm consensus requirements.
SL-2	Configurable Consensus with Hard Minimums	Consensus thresholds are mission-configurable (e.g., simple majority, supermajority, unanimous), but hardcoded minimums prevent any mission configuration from reducing consensus below safety-critical thresholds.
SL-3	Graceful Degradation Under Agent Loss	Swarm coherence is maintained even under partial agent loss. The swarm recalculates quorum requirements and redistributes responsibilities. Degradation is graceful, not catastrophic.
SL-4	Comms-Denied Default	When communication between agents is denied or degraded, each agent defaults to the last-known-good doctrine state. No agent improvises governance rules in the absence of swarm connectivity.

3.3 Consensus and Escalation Protocol

The Swarm Law consensus mechanism operates through a structured escalation flow:

10. **Proposal** — An agent proposes an action or state change to the swarm.
11. **Evaluation** — All reachable agents evaluate the proposal against their local doctrine state and mission context.
12. **Vote** — Agents submit compliance/non-compliance votes with rationale.
13. **Threshold Check** — The Doctrine Module evaluates whether the consensus threshold has been met.
14. **Execution or Escalation** — If threshold is met, the action proceeds. If not, the proposal is escalated: first to a higher-priority agent subset, then to the operator interface if communication permits.
15. **Deadlock Resolution** — If escalation cannot resolve the conflict (e.g., comms-denied), the system defaults to the most conservative action consistent with the current degradation level.

[DIAGRAM: Swarm Law — Consensus and Escalation Flow]

This diagram should depict the six-step consensus flow as a vertical flowchart. Each step is a labeled box. Decision diamonds at steps 4 and 5 branch to "Execute" (threshold met) or "Escalate" (threshold not met). The escalation path shows three tiers: peer agents → priority agent subset → operator interface. A final branch from "Operator Unreachable" leads to "Default to Conservative Action." Annotate the hardcoded minimum consensus threshold at the Threshold Check step.

3.4 Mission Relevance

Swarm Law enables several mission-critical operational scenarios that are infeasible under centralized control architectures:

- **Multi-Rover Planetary Surface Coordination** — Multiple rovers operating on a planetary surface can coordinate exploration, sample collection, and hazard avoidance without requiring individual ground commands for each unit. Swarm Law ensures that no single rover can unilaterally redirect the team's mission objectives.
- **Distributed Sensor Networks in Deep Space** — Constellations of sensor platforms operating beyond Earth orbit can collectively decide on observation priorities, data relay routing, and power management through governed consensus rather than ground-scheduled command sequences.
- **Cooperative Robotics in Orbital Construction** — Multi-agent robotic systems performing orbital assembly tasks can negotiate subtask allocation, sequence dependencies, and safety interlocks through Swarm Law — maintaining construction integrity even if individual units fail.

Key Takeaway

Swarm Law replaces centralized ground control with governed collective autonomy. Agents coordinate through codified consensus rules — not ad-hoc negotiation — ensuring that swarm behavior remains predictable and auditable even in communication-denied environments.

4. Pillar II — Identity and Role Enforcement

4.1 Purpose

The Identity and Role Enforcement system ensures that every agent in the AAIS architecture maintains a stable, verifiable identity and operates strictly within its assigned role boundaries. This pillar addresses a class of failure modes that are unique to multi-agent systems: identity confusion, role drift, and capability assumption — where an agent begins performing functions outside its design envelope, potentially with catastrophic results.

4.2 Core Mechanisms

ID	Mechanism	Definition
IR-1	Identity Anchoring	Each agent's identity is cryptographically bound at instantiation. The identity anchor includes the agent's type, version, capability manifest, and role assignment. This anchor cannot be self-modified by the agent — identity changes require operator authorization through the governance protocol.
IR-2	Role Boundary Enforcement	Agents cannot assume capabilities or permissions outside their defined role, even under pressure, novel conditions, or explicit requests from other agents. Role boundaries are enforced at the Invariant Engine level and cannot be circumvented by agent-level logic.

ID	Mechanism	Definition
IR-3	Impersonation Prevention	No agent may claim another agent's identity or role. All inter-agent communications include cryptographic identity attestation. Messages that fail identity verification are rejected and logged as governance events.
IR-4	Continuous Verification	Identity verification is continuous, not one-time. Agents are re-verified at governance checkpoints (periodic audits and event-triggered evaluations). An agent that fails re-verification is isolated from the swarm pending investigation.

4.3 Identity Lifecycle

The identity lifecycle of an AAIS agent follows a governed sequence:

16. **Instantiation** — Agent is created with a cryptographic identity anchor, capability manifest, and initial role assignment.
17. **Registration** — Agent registers with the Doctrine Module and receives its governance profile for the current mission phase.
18. **Active Operation** — Agent executes within role boundaries, subject to continuous identity verification at governance checkpoints.
19. **Role Modification** (if required) — Role changes are proposed through the governance protocol, evaluated against doctrine invariants, and require operator authorization when communication permits.
20. **Decommission** — Agent identity is revoked and archived. The swarm recalculates quorum and redistributes responsibilities per Swarm Law.

[DIAGRAM: Identity Anchor and Role Boundary Architecture]

This diagram should show an individual agent's identity structure as a layered block: Cryptographic Identity Anchor (base layer), Capability Manifest (middle layer), and Role Assignment (top layer). Surrounding the block: the Role Boundary enforcement perimeter, depicted as a hard boundary line. Arrows showing inter-agent communication should pass through an "Identity Attestation Gateway" before reaching other agents. A separate callout should show the continuous verification cycle: Agent → Governance Checkpoint → Re-Verification → Continue/Isolate.

4.4 Mission Relevance

Identity and Role Enforcement is critical for preventing cascading failure modes in multi-agent space systems:

- **Cross-Domain Interference Prevention** — A science instrument agent cannot override navigation functions. A communications agent cannot modify mission parameters. Each agent is confined to its domain by architectural enforcement, not behavioral convention.
- **Malfunction Containment** — If an agent begins exhibiting anomalous behavior due to hardware degradation or software fault, Identity Enforcement ensures that the malfunctioning agent cannot expand its operational scope. It is contained within its role boundaries and, if it fails continuous verification, isolated from the swarm.
- **Multi-Organization Operations** — In missions involving agents from multiple organizations or development teams, Identity Enforcement provides a trust framework that does not depend on uniform software provenance. Each agent proves its identity and role cryptographically.

Key Takeaway

Identity is not self-reported — it is cryptographically anchored and continuously verified. Role boundaries are enforced at the architectural level. An agent that cannot prove its identity or exceeds its role is automatically isolated, preventing cascading failures across the multi-agent system.

5. Pillar III — Degradation Doctrine

5.1 Purpose

The Degradation Doctrine defines how the AAIS system behaves as components fail, degrade, or lose connectivity. Its central mandate is that the system must always transition to a **known-safe state** rather than an undefined or emergent one. In mission-critical space systems, undefined behavior during failure is unacceptable — the Degradation Doctrine ensures that every failure mode has a governed response.

5.2 Core Principles

ID	Principle	Definition
DD-1	Defined Degradation Paths	Every operational mode has a pre-defined degradation path specifying the system's behavior when that mode can no longer be sustained.
DD-2	Governed, Not Emergent	Degradation follows pre-defined fallback chains. The system does not improvise responses to failure — it executes the degradation path defined for the specific failure

ID	Principle	Definition
		signature.
DD-3	Fail-to-Safe Mandatory	"Fail-to-safe" is the only acceptable failure behavior. "Fail-to-silent" — where the system stops responding without transitioning to a safe state — is explicitly prohibited by doctrine.
DD-4	Mandatory Event Logging	All degradation events are logged with timestamps, affected agent IDs, failure signatures, and the degradation path executed. Logs are persisted to non-volatile storage and reported at the earliest communication opportunity.
DD-5	Defined Recovery Paths	Recovery paths are defined alongside degradation paths. The system knows how to restore higher operational modes when conditions improve — not only how to degrade.

5.3 Tiered Degradation Model

The Degradation Doctrine defines five operational tiers with explicit transition criteria:

Tier	State	Conditions	System Behavior
0	**Nominal**	All subsystems operational. Full communication. All agents verified and within role boundaries.	Full autonomous operation per mission plan. All capabilities active.

Tier	State	Conditions	System Behavior
1	**Caution**	Minor anomalies detected. Non-critical subsystem degradation. Communication latency within expected bounds.	Continued operation with increased monitoring frequency. Non-essential tasks may be deferred. Anomaly reported to operator.
2	**Degraded**	One or more critical subsystems impaired. Partial agent loss. Communication intermittent or significantly delayed.	Mission objectives reprioritized. Non-critical agents suspended. Swarm quorum recalculated. Operator notified at first opportunity.
3	**Safe-Hold**	Multiple critical failures. Communication denied. Swarm coherence below minimum threshold.	All non-essential operations halted. System maintains minimum safe configuration. Agents execute last-known-good doctrine. Continuous attempt to re-establish communication.
4	**Emergency Shutdown**	Unrecoverable system failure. Hardware integrity compromised. Continued operation poses risk to mission assets or personnel.	Controlled shutdown sequence. All state persisted to non-volatile storage. Final telemetry burst transmitted. System enters lowest-power safe state.

Transitions between tiers are governed by explicit criteria — not subjective assessment. Each transition requires the concurrence of the Doctrine Module and the Invariant Engine. Upward transitions (recovery) follow the same governed process as downward transitions (degradation).

[DIAGRAM: Degradation Doctrine — Tiered State Transitions]

This diagram should depict the five operational tiers as a vertical state machine. Each tier is a labeled box color-coded per the table above. Downward arrows (degradation) are labeled with transition criteria. Upward arrows (recovery) are shown as dashed lines with recovery criteria. A side annotation should indicate that all transitions are logged and that "fail-to-silent" transitions (skipping states) are prohibited. The Emergency Shutdown state should be visually distinct (bold border) with a note that it requires controlled shutdown sequence completion.

5.4 Mission Relevance

The Degradation Doctrine directly addresses NASA's operational requirements for fault-tolerant autonomy:

- **Mars Surface Operations** — One-way communication delay of 4–24 minutes means ground controllers cannot intervene in real-time failures. The Degradation Doctrine ensures that the system's response to cascading hardware failures is pre-defined, predictable, and auditable — not emergent.
- **Outer Planet Missions** — Communication delays of 35–85+ minutes to Jupiter and Saturn make ground intervention impractical for any time-critical failure response. The tiered degradation model ensures that the system can manage its own failure states across extended communication blackouts.
- **Entry, Descent, and Landing (EDL)** — During EDL phases, communication is often interrupted entirely. The Degradation Doctrine provides pre-defined fallback chains for every EDL contingency, ensuring the system transitions to safe states even when ground contact is impossible.

Key Takeaway

The Degradation Doctrine eliminates undefined failure states. Every degradation path leads to a known-safe state. Every recovery path is pre-defined. "Fail-to-silent" is never acceptable. The system's behavior under failure is as governed and auditable as its behavior under nominal conditions.

6. Invariant Engine — Constraint Enforcement Layer

The Invariant Engine is the formal verification subsystem that evaluates every agent action against codified, non-negotiable system truths before execution. Where the Doctrine Module defines the governance laws, the Invariant Engine enforces them at runtime — serving as the final gate between agent intent and system actuation.

6.1 Purpose

The Invariant Engine defines and enforces hard physical, operational, and mission-level constraints that no agent, governance override, or environmental condition can violate. These are not soft preferences or tunable parameters — they are absolute boundaries derived from the laws of physics, hardware specifications, and mission-critical safety thresholds.

6.2 Constraint Categories

ID	Category	Description	Example Invariants
INV-1	Thermal Limits	Maximum and minimum operating temperatures for all hardware components.	"Component temperature must not exceed 85°C"; "Heater activation required below -40°C"

ID	Category	Description	Example Invariants
		Actions that would cause thermal exceedance are blocked before execution.	
INV-2	Structural Limits	Load-bearing thresholds, vibration tolerances, and mechanical stress limits. The engine prevents actions that would exceed structural design margins.	"Robotic arm load must not exceed 15kg"; "Deployment torque within rated envelope"
INV-3	Power Reserve	Minimum power reserves required to maintain safe-hold state and communication capability. No action may reduce power below the reserve threshold.	"Battery state-of-charge must remain above 20%"; "Solar array orientation must maintain minimum charge rate"
INV-4	Mission Viability	High-level constraints ensuring that autonomous decisions do not compromise overall mission objectives or timeline-critical milestones.	"Sample cache integrity must be preserved"; "Communication window obligations must be met"

6.3 Enforcement Model

The Invariant Engine operates as a synchronous gate in the action execution pipeline. Every proposed agent action is serialized into a constraint evaluation request. The engine evaluates the action against all applicable invariants and returns one of three verdicts:

- **PASS** — The action satisfies all applicable invariants. Execution proceeds.

- **BLOCK** — The action would violate one or more invariants. Execution is prevented. The violation is logged with the specific invariant(s) that would be breached and the margin of violation.
- **CONDITIONAL** — The action satisfies invariants only under specific conditions (e.g., reduced power mode, thermal pre-conditioning). The conditions are returned to the Doctrine Module for evaluation.

Key Takeaway

The Invariant Engine pairs with the Doctrine Module to create a two-layer governance architecture: doctrine defines the behavioral laws, invariants enforce the physical and operational boundaries. Together, they ensure that no agent action can violate either governance policy or the laws of physics — regardless of how novel or urgent the operational context.

7. Governance Integration and Auditability

The Doctrine Module is designed for seamless integration with NASA's existing mission assurance frameworks and verification processes. Governance is not an afterthought — it is the system's primary architectural concern.

7.1 Comprehensive Event Logging

All doctrine enforcement events are logged with the following data fields:

Field	Description
Timestamp	UTC timestamp of the governance event, synchronized across all agents via onboard clock reconciliation.
Event Type	Classification of the event (e.g., compliance check, violation, escalation,

Field	Description
	degradation transition, identity verification).
Agent ID(s)	Cryptographic identity of the agent(s) involved in the event.
Decision Rationale	Machine-readable explanation of the governance decision, including the doctrine rule invoked and the invariant evaluation result.
Outcome	Result of the governance evaluation (permitted, blocked, escalated, agent isolated).
Context Snapshot	Relevant system state at the time of the event, including degradation tier, swarm composition, and communication status.

7.2 Real-Time and Post-Mission Audit

Doctrine compliance can be audited through two complementary channels:

- **Real-Time Operator Dashboard** — When communication permits, operators can monitor doctrine compliance in real time via the operator interface. The dashboard provides governance state visualization, active violation alerts, and escalation queue management.
- **Post-Mission Audit** — Complete governance logs are available for post-mission review. Logs support forensic analysis of governance decisions, enabling reviewers to trace any system action back to the doctrine rule that authorized or blocked it.

7.3 Mission Phase Governance Profiles

The Doctrine Module supports configurable governance profiles for different mission phases. Each profile adjusts consensus thresholds, degradation tier criteria, and role permissions to match the operational requirements of the current phase:

Mission Phase	Governance Profile Characteristics
Launch	Highest-restriction governance. Minimal autonomous authority. All non-essential agent actions deferred. Ground override priority.
Cruise	Moderate autonomous authority. Routine operations delegated to agents. Ground-scheduled governance checkpoints.
EDL	Full autonomous authority with maximum governance enforcement. Pre-defined contingency chains active. No ground override expected.
Surface Operations	Standard autonomous authority. Swarm Law fully active. Governance profile adjustable by ground operators between planning cycles.

7.4 V&V Compatibility

The Doctrine Module is designed for compatibility with NASA's Verification and Validation (V&V) processes:

- **Formal Specification** — All doctrine invariants are expressed in a formal specification language amenable to automated verification.
- **Test Harness Integration** — The Doctrine Module exposes a test interface that allows V&V teams to inject governance scenarios and verify system responses against expected outcomes.
- **Traceability** — Every doctrine rule is traceable to a mission requirement, enabling requirements-to-governance mapping in standard V&V documentation.
- **Deterministic Evaluation** — Doctrine compliance evaluation is deterministic: the same inputs always produce the same governance decision, enabling reproducible testing.

Key Takeaway

Every governance decision is logged, traceable, and auditable. The Doctrine Module produces a complete chain of evidence from mission requirement → doctrine rule → runtime enforcement → outcome — enabling full accountability in post-mission review and V&V processes.

8. Comparative Advantage

The AAIS Doctrine Module occupies a distinct position in the autonomous systems landscape, addressing limitations present in both traditional rule-based approaches and modern machine learning systems.

Approach	Adaptability	Behavioral Guarantees	Auditability
Traditional Rule-Based Systems	Low — behavior is fully pre-scripted. Cannot adapt to conditions outside the rule set.	High — behavior is deterministic and verifiable, but only for anticipated scenarios.	High — every action traces to a rule.
Pure ML/AI Systems	High — can generalize to novel conditions through learned representations.	Low — behavior under novel conditions is not guaranteed. Verification is statistical, not formal.	Low — decision rationale is often opaque.
AAIS Doctrine Module	High — agents can learn and adapt within governance boundaries. Novel responses are permitted if they satisfy doctrine invariants.	High — behavioral invariants are formally specified and enforced at runtime. Guarantees hold even under novel conditions.	High — every governance decision is logged with full rationale and context.

The AAIS Doctrine Module provides **adaptive autonomy with codified behavioral invariants**. The system can learn, generalize, and respond to novel conditions — but it cannot violate its governance laws in doing so. This combination

addresses NASA's dual requirement for autonomous systems that are both capable enough to operate without real-time human oversight and predictable enough to satisfy mission assurance requirements.

> **Note**
>
> The Doctrine Module does not replace ML/AI capabilities — it governs them. Agents may use learned models for perception, planning, and decision-making. The Doctrine Module ensures that the outputs of those models are compliant with governance invariants before they are actuated.

9. Conclusion and Recommended Next Steps

The AAIS Doctrine Module v2.1 provides a governance framework purpose-built for autonomous systems operating in environments where traditional ground-controlled paradigms are insufficient. Its three pillars — Swarm Law, Identity and Role Enforcement, and Degradation Doctrine — collectively ensure that multi-agent autonomous systems remain predictable, auditable, and safe under the full range of conditions encountered in space missions.

The module's architecture enforces governance at the foundational level, integrates with NASA's existing V&V processes, and provides comprehensive auditability through deterministic event logging and traceable decision rationale.

9.1 Recommended Phased Integration

We recommend a three-phase evaluation and integration approach:

Phase	Activity	Duration (Est.)	Description
1	**Simulation-Based Evaluation**	6–9 months	Deploy the Doctrine Module within NASA's existing simulation

Phase	Activity	Duration (Est.)	Description
			testbeds. Evaluate governance behavior across representative mission scenarios including nominal operations, degraded communications, and cascading failures. Validate V&V integration.
2	**Hardware-in-the-Loop Testing**	9–12 months	Integrate the Doctrine Module with representative flight hardware. Evaluate real-time governance enforcement performance, identity verification latency, and degradation doctrine response times under realistic hardware constraints.
3	**Technology Demonstration Mission**	12–18 months	Pilot deployment on a technology demonstration mission with limited operational scope. Evaluate end-to-end governance performance in an actual space environment. Collect flight data for post-mission audit and doctrine refinement.

9.2 Points of Contact

For technical inquiries, simulation testbed access requests, and integration planning, please contact the Cognitive Operating Systems Division, AAIS Platform. Detailed technical specifications, API documentation, and formal invariant definitions are available upon request under appropriate access agreements.

Key Takeaway

The AAIS Doctrine Module v2.1 is ready for structured NASA evaluation. The recommended phased approach — simulation, hardware-in-the-loop, technology demonstration — provides a low-risk path to validating governed autonomy for mission-critical space systems.
